

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar	
		Ms. B. Swathi	
		Dr. Sasanko Shekhar Gantayat	
		Mr. Md Sallauddin	
		Dr. Mathivanan	
		Mr. Y Srikanth	
		Ms. N Shilpa	
		Dr. Rishabh Mittal (Coordinator)	
		Dr. R. Prashant Kumar	
		Mr. Ankushavali MD	
		Mr. B Viswanath	
		Ms. Sujitha Reddy	
		Ms. A. Anitha	
		Ms. M. Madhuri	
		Ms. Katherashala Swetha	
		Ms. Velpulasumalatha	
		Mr. Bingi Raju	
Course Code	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week 6 - Tuesday	Time(s)	
Duration	2 Hours	Applicable to Batches	23CSBTB01 To 23CSBTB52
Assignment Number: 11.1 (Present assignment number) / 24 (Total number of assignments)			
Q.No.	Question		Expected Time to complete
1	Lab 11 – Data Structures with AI: Implementing Fundamental Structures Lab Objectives - Use AI to assist in designing and implementing fundamental data structures in Python. - Learn how to prompt AI for structure creation, optimization, and documentation.		Week 6 - Tuesday

- Improve understanding of Lists, Stacks, Queues, Linked Lists, Trees, Graphs, and Hash Tables.
- Enhance code quality with AI-generated comments and performance suggestions.

Task Description -1 – (Stack Using AI Guidance)

- **Task:** With the help of AI, design and implement a Stack data structure supporting basic stack operations.

Expected Output:

- A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.

Prompt

Design a Python program to implement a **Stack data structure** with basic operations such as push, pop, peek, and isEmpty.

Code

```

AI ASSISTANT
day2.py
day3.py
day3.py > Stack > is_empty
1 # Stack Implementation in Python
2 class Stack:
3     def __init__(self):
4         self.items = []
5
6     # Push element into stack
7     def push(self, item):
8         self.items.append(item)
9         print(item, "pushed into stack")
10
11    # Pop element from stack
12    def pop(self):
13        if self.is_empty():
14            return "Stack is empty"
15        return self.items.pop()
16
17    # Peek top element
18    def peek(self):
19        if self.is_empty():
20            return "Stack is empty"
21        return self.items[-1]
22
23    # Check if stack is empty
24    def is_empty(self):
25        return len(self.items) == 0
26
27    # Driver Code
28    s = Stack()
29    s.push(10)
30    s.push(20)
31    s.push(30)
32    print("Top element:", s.peek())
33    print("Popped element:", s.pop())
34    print("Stack empty?", s.is_empty())
35
OUTLINE

```

Output

```
PS C:\Users\konda\Downloads\AI Assistant> & C:\Users\konda\AppData\Local\Microsoft\Windows\SelfUpdate\WindowsUpdate\WindowsUpdate.exe --uninstall --quiet --no-reboot --no-log
10 pushed into stack
20 pushed into stack
30 pushed into stack
Top element: 30
Popped element: 30
Stack empty? False
PS C:\Users\konda\Downloads\AI Assistant> █
```

Explanation

- A Stack follows LIFO (Last In First Out) principle.
- The program uses a list to store stack elements.
- push() → adds an element to the top
- pop() → removes the top element
- peek() → shows the top element without removing
- is_empty() → checks whether the stack is empty

Task Description -2 – (Queue Design)

- **Task:** Use AI assistance to create a Queue data structure following FIFO principles

Expected Output:

- A complete Queue implementation including enqueue, dequeue, front element access, and size calculation

Prompt

Design a Python program to implement a **Queue data structure** that follows the **FIFO (First In First Out)** principle.

Code

```

1  # Queue Implementation in Python
2  class Queue:
3      def __init__(self):
4          self.items = []
5          # Add element to queue
6      def enqueue(self, item):
7          self.items.append(item)
8          print(item, "enqueued")
9          # Remove element from queue
10     def dequeue(self):
11         if self.is_empty():
12             return "Queue is empty"
13         return self.items.pop(0)
14     # Get front element
15     def front(self):
16         if self.is_empty():
17             return "Queue is empty"
18         return self.items[0]
19     # Get size of queue
20     def size(self):
21         return len(self.items)
22     # Check if empty
23     def is_empty(self):
24         return len(self.items) == 0
25     # Driver Code
26     q = Queue()
27     q.enqueue(10)
28     q.enqueue(20)
29     q.enqueue(30)
30     print("Front element:", q.front())
31     print("Dequeued element:", q.dequeue())
32     print("Queue size:", q.size())

```

Output

```

PS C:\Users\konda\Downloads\AI Assistant> & C:\Users\konda\AppData\Local\Microsoft\WindowsApps
10 enqueued
20 enqueued
30 enqueued
Front element: 10
Dequeued element: 10
Queue size: 2
PS C:\Users\konda\Downloads\AI Assistant>

```

Explanation

- A Queue follows FIFO (First In First Out).
- The program uses a Python list to store elements.
- enqueue() → adds element at rear
- dequeue() → removes element from front
- front() → shows first element without removing
- size() → returns number of elements

Task Description -3 –(Singly Linked List Construction)

- **Task:** Utilize AI to build a singly linked list supporting insertion and traversal.

Expected Output:

- Correctly functioning linked list with node creation, insertion

logic, and display functionality.

Prompt

Implement singly linked list with insertion and traversal.

Code

```
day3.py > ...
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class LinkedList:
7     def __init__(self):
8         self.head = None
9
10    def insert(self, data):
11        new = Node(data)
12        if not self.head:
13            self.head = new
14            return
15        temp = self.head
16        while temp.next:
17            temp = temp.next
18        temp.next = new
19
20    def display(self):
21        temp = self.head
22        while temp:
23            print(temp.data, end=" -> ")
24            temp = temp.next
25
26
27 ll = LinkedList()
28 ll.insert(10); ll.insert(20); ll.insert(30)
29 ll.display()
```

Output

```
PS C:\Users\konda\Downloads\AI Assistant> & C:\Users\konda\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:/Users/konda/Downloads/day3.py"
10 -> 20 -> 30 ->
PS C:\Users\konda\Downloads\AI Assistant> |
```

Explanation

Nodes are connected sequentially using pointers. Traversal prints all elements.

Task Description -4 – (Binary Search Tree Operations)

- **Task:** Implement a Binary Search Tree with AI support focusing on insertion and traversal.

Expected Output:

- BST program with correct node insertion and in-order traversal output.

Prompt

Implement BST with insertion and in-order traversal.

Code

```
day3.py > ...
1 class BSTNode:
2     def __init__(self, data):
3         self.data = data
4         self.left = None
5         self.right = None
6
7     def insert(root, key):
8         if not root:
9             return BSTNode(key)
10        if key < root.data:
11            root.left = insert(root.left, key)
12        else:
13            root.right = insert(root.right, key)
14        return root
15
16    def inorder(root):
17        if root:
18            inorder(root.left)
19            print(root.data, end=" ")
20            inorder(root.right)
21
22    root = None
23    for x in [50, 30, 70, 20, 40, 60, 80]:
24        root = insert(root, x)
25
26    inorder(root)
27
```

Output

```
PS C:\Users\konda\Downloads\AI Assistant> & C:\Users\konda\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.9.0.0_qblws89re1wje\python.exe day3.py
20 30 40 50 60 70 80
PS C:\Users\konda\Downloads\AI Assistant>
```

Explanation

BST maintains sorted order. In-order traversal prints elements in ascending order.

Task Description -5 – (Hash Table Implementation)

- **Task:** Create a hash table using AI with collision handling

Expected Output:

- Hash table supporting insert, search, and delete using chaining

or open

Prompt

Create a hash table with collision handling using chaining.

Code

```
1 class HashTable:
2     def __init__(self, size):
3         self.size = size
4         self.table = [[] for _ in range(size)]
5
6     def hash(self, key):
7         return key % self.size
8
9     def insert(self, key):
10        idx = self.hash(key)
11        self.table[idx].append(key)
12
13    def search(self, key):
14        idx = self.hash(key)
15        return key in self.table[idx]
16
17    def delete(self, key):
18        idx = self.hash(key)
19        if key in self.table[idx]:
20            self.table[idx].remove(key)
21
22
23 h = HashTable(5)
24 h.insert(10); h.insert(15); h.insert(20)
25 print(h.search(15))
26 h.delete(15)
27 print(h.search(15))
```

Output

```
● PS C:\Users\konda\Downloads\AI Assistant> & C:\Users\konda\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.9.0.0_qblz94qz8qbf\python.exe C:\Users\konda\Downloads\AI Assistant\hash_table.py
True
False
○ PS C:\Users\konda\Downloads\AI Assistant> █
```

Explanation

Hash function maps keys to index. Collisions handled using list chaining.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.